



Retinal Review Workbench

Retinal Review Workbench

2026-09-23

Table of contents

1	Deployment and Operations Manual	3
1.1	Document control	3
2	Architecture and supported scope	4
2.1	Topology	4
2.2	Boundaries	5
2.3	Data ownership	5
3	Installation	6
3.1	Review workstation	6
3.2	Linux server checkout	6
3.3	Configuration	6
4	Model API setup and startup	7
4.1	One-time online setup	7
4.2	Normal startup	7
4.3	Health and model metadata	7
4.4	Offline-after-setup	8
5	Routine operations	9
5.1	systemd service	9
5.2	Connect a workstation	9
5.3	Daily readiness check	9
6	Security, privacy, backup, and restore	10
6.1	Security and privacy	10
6.2	Backup	10
6.3	Restore	10
6.4	CVAT	10
7	Upgrade, rollback, and release acceptance	11
7.1	Upgrade	11
7.2	Rollback	11
7.3	Troubleshooting	11
7.4	Release acceptance	12

1

Deployment and Operations Manual



Software version 0.7.0

Manual version 1.0

Controlled release: 2026-09-23

This manual covers the supported review workstation and hospital Model API deployment. It is written for hospital IT operators and maintainers. It does not replace the [Clinician User Manual](#).

1.1 Document control

Field	Value
Document ID	RRW-OPS-UM-001
Software version	0.7.0
Manual version	1.0
Status	Controlled release documentation
Date	2026-09-23
Primary audience	Hospital IT, deployment operators, maintainers

WARNING - protected deployment

Keep the Model API on a protected hospital network. Do not expose it directly to the public internet. Apply hospital firewall, TLS, authentication, and secret-management policy at the deployment boundary.

2

Architecture and supported scope

2.1 Topology

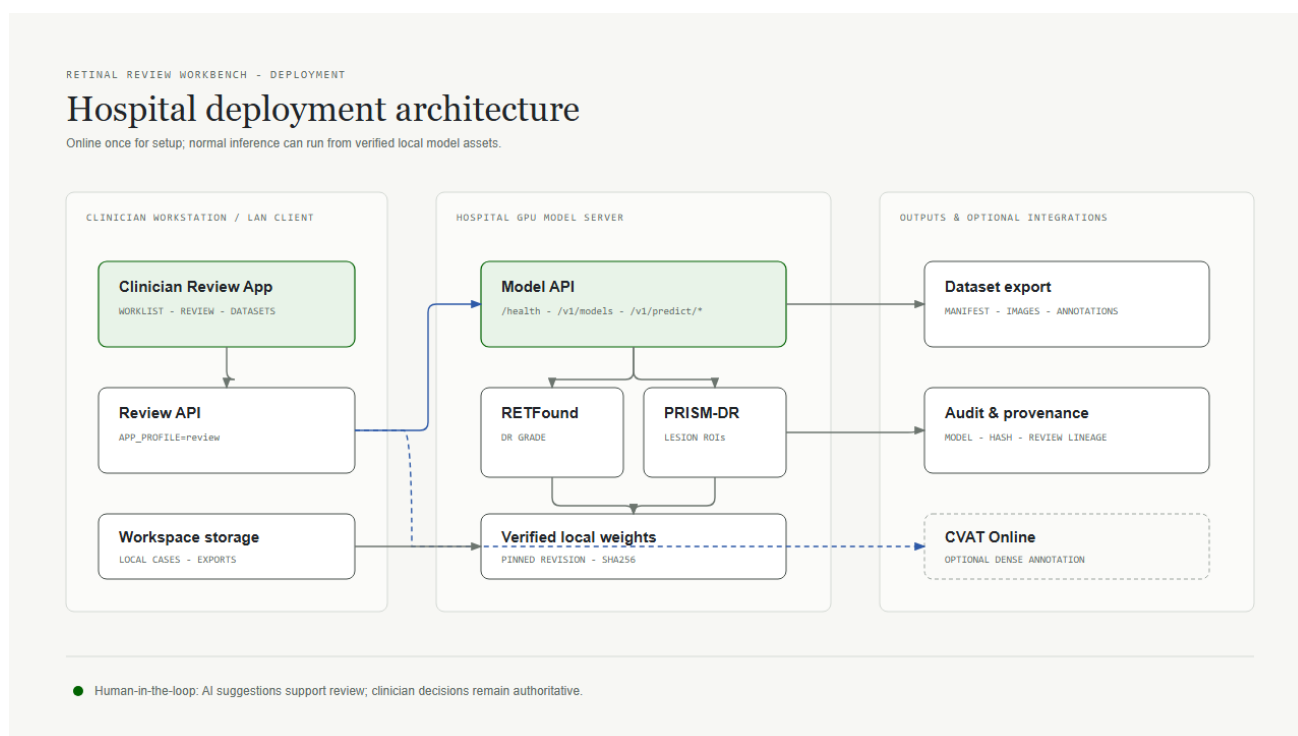


Figure 2.1: Hospital deployment architecture

Component	Role
Review workstation	React/Vite clinician UI, FastAPI review routes, Workspace catalog, local review state, annotations, and export.
Model API	FastAPI service on a GPU host with APP_PROFILE=model_api and MODEL_RUNTIME=local.
RETFound	Five-class DR grade suggestion through retfound-aptos5.
PRISM-DR	Optional lesion ROI suggestions through prism-dr-5fold.

2.2 Boundaries

The review profile does not load model weights. It calls the provider-neutral Model API. The Model API supports the existing fundus image contracts only. MRI, OCT, PACS, DICOMweb, calibration, XAI/Grad-CAM, and autonomous diagnosis are outside this release.

The application retains the `/ui` static surface for compatibility with existing local integrations. Normal users and deployments must use `/app/`; `/ui` is legacy compatibility only.

2.3 Data ownership

The review workstation owns Workspace configuration, case state, annotations, audit events, and export files. The Model API is stateless for clinical records and returns model metadata, inference results, and provenance. Original admitted source images remain immutable.

3

Installation

3.1 Review workstation

Prerequisites: Windows or Linux host, Python 3.11 or 3.12, Node.js/npm, and optional DICOM extras when DICOM input is required.

```
git clone https://github.com/siriponsri/dr-support-screening-poc.git
cd dr-support-screening-poc
py -3.12 -m venv .venv
.venv\Scripts\python.exe -m pip install --upgrade pip
.venv\Scripts\python.exe -m pip install -e ".[test,dicom]"
npm ci
cd frontend
npm ci
npm run build
cd ..
START.cmd
```

Open `http://127.0.0.1:8000/app/`. The review launcher uses `APP_PROFILE=review` and `MODEL_RUNTIME=remote`. Set `REMOTE_MODEL_URL` and the optional `REMOTE_MODEL_TOKEN` outside Git before starting when a Model API is available.

3.2 Linux server checkout

Place the release at `/opt/dr-support-screening-poc` when using the included systemd unit. Keep runtime assets in `local-state/` or another private path. Never commit model weights, review databases, or secret environment files.

3.3 Configuration

Copy `.env.example` only as a reference. Use private process environment or an approved secret manager for tokens. The model-server example is [deployment/model-server.env.example](#). See [Configuration](#) for the complete variable matrix.

4

Model API setup and startup

4.1 One-time online setup

Run on the supported Linux NVIDIA GPU host:

```
cd /opt/dr-support-screening-poc
./scripts/model-server/setup.sh
```

The setup script creates `.venv`, installs `[models,dicom]`, acquires the pinned RETFound and PRISM-DR sources and checkpoint assets, verifies revisions and SHA-256 values, checks the configured CUDA device, and runs synthetic smoke inference. It prints READY only after verification and smoke inference succeed.

Pinned source and asset facts are maintained in [docs/MODEL_SERVER.md](#), `dr_support/providers/retfound.py`, `dr_support/providers/prism.py`, and `dr_support/providers/prism_assets.json`.

4.2 Normal startup

```
cd /opt/dr-support-screening-poc
./scripts/model-server/start.sh
```

Normal startup sets `APP_PROFILE=model_api`, `MODEL_RUNTIME=local`, `INFERENCE_DEVICE=cuda:0`, `HOST=0.0.0.0`, `PORT=7860`, and `WORKERS=1` unless the operator overrides the documented values. It runs read-only verification with `--verify`; it does not download or silently replace assets.

4.3 Health and model metadata

```
./scripts/model-server/healthcheck.sh
curl --fail http://127.0.0.1:7860/health
curl --fail http://127.0.0.1:7860/v1/models
```

Expected readiness is `/health` with `"status":"PASS"` and verified assets, followed by `/v1/models` containing `retfound-aptos5` and `prism-dr-5fold`.

4.4 Offline-after-setup

After the one-time setup succeeds, disconnect external network access and restart with `start.sh`. Readiness and local inference use only verified local source checkouts and checkpoint files. CVAT Online is optional and excluded from offline acceptance.

CAUTION - fail closed

If a source revision, checkpoint hash, CUDA device, or required asset is missing or modified, startup must fail and require operator intervention. Do not bypass verification by switching silently to another model or device.

5

Routine operations

5.1 systemd service

Install the included service only when the release is at `/opt/dr-support-screening-poc`:

```
cd /opt/dr-support-screening-poc
sudo ./scripts/model-server/install-service.sh
sudo systemctl start dr-support-model-api
sudo systemctl status dr-support-model-api
./scripts/model-server/healthcheck.sh
```

The service runs as `drsupport`, reads `/etc/dr-support/model-server.env`, uses one worker, and starts `scripts/model-server/start.sh`.

Use the following for lifecycle and logs:

```
sudo systemctl stop dr-support-model-api
sudo systemctl restart dr-support-model-api
sudo journalctl -u dr-support-model-api -n 100 --no-pager
```

5.2 Connect a workstation

Set `REMOTE_MODEL_URL` on the review workstation to the Model API base URL, for example `http://model-host.example:7860`. Use HTTPS across an untrusted segment. Set `REMOTE_MODEL_TOKEN` through a private environment or secret manager when required. The token is not returned to the UI, stored in SQLite, or logged.

5.3 Daily readiness check

1. Confirm the service is active with `systemctl status`.
2. Run `healthcheck.sh` and confirm `/health` reports PASS.
3. Confirm `/v1/models` returns both expected model IDs and verified status.
4. Open the review workstation and check **Models & Audit**.
5. Run a public or synthetic smoke image only when the release acceptance procedure requires it.

6

Security, privacy, backup, and restore

6.1 Security and privacy

Use approved public, synthetic, or authorized clinical data. Keep PHI, private DICOM headers, model weights, tokens, passwords, CVAT credentials, runtime SQLite files, and local-state directories out of Git and screenshots. Original source images are immutable.

The product is a research and clinical-support proof of concept. Human review remains authoritative. No product claim establishes regulatory approval, autonomous diagnosis, referral, treatment, or calibrated probability.

6.2 Backup

Back up the active Workspace SQLite database, approved export packages, Workspace configuration, and the exact release/configuration record according to hospital policy. Keep model assets with their verified hashes and preserve the relationship between release, model assets, and exports.

Do not back up or copy secret files into the repository. Store `/etc/dr-support/model-server.env` through the hospital secret-management process.

6.3 Restore

1. Stop the affected service or workstation process.
2. Restore the Workspace database and approved export package to a controlled path.
3. Restore the same release or a validated compatible release.
4. Verify file ownership and permissions.
5. Run the model asset verification and `/health` checks.
6. Open the workstation, inspect the Workspace, and confirm source and annotation hashes before resuming review.

6.4 CVAT

CVAT Online is optional. Keep `CVAT_TOKEN` only in secure runtime configuration. Do not paste it into commands, browser JavaScript, screenshots, documentation, or issue reports. Ambiguous remote writes require manual reconciliation; never retry blindly.

7

Upgrade, rollback, and release acceptance

7.1 Upgrade

1. Keep the previous validated release and model asset directory intact.
2. Place the new clean checkout at a separate path.
3. Run the one-time model setup on the new release if its verified assets are not already present.
4. Run backend tests, Ruff, frontend tests, typecheck, and production build.
5. Start the new Model API, run health and model metadata checks, and perform one public/synthetic smoke inference.
6. Connect a review workstation and verify `/app/`, dataset export, and Models & Audit.
7. Record the release SHA, software version, model asset hashes, and acceptance result before switching service configuration.

7.2 Rollback

Stop the service, restore the previous release/configuration, verify the previous asset hashes, and restart. Do not overwrite verified model assets in place. Preserve the failed release for investigation according to hospital policy.

7.3 Troubleshooting

Observation	Action
READY is not printed	Read the setup error. Check Python, dependencies, CUDA, source revisions, checkpoint hashes, and storage.
Startup fails verification	Treat changed or missing assets as a deliberate fail-closed state. Reacquire through the controlled online setup.
<code>/health</code> is not PASS	Check service logs, CUDA visibility, environment values, port binding, and asset verification.
<code>/v1/models</code> is missing a model	Check local assets and the configured provider package. Do not substitute a different model.

Observation	Action
Workstation cannot infer	Check <code>REMOTE_MODEL_URL</code> , token handling, firewall policy, and Model API health. Human review and export can remain available.
DICOM is unsupported	A non-fundus DICOM should be reported as Unsupported modality . Do not expand support to MRI, OCT, PACS, or DICOMweb.

7.4 Release acceptance

Record the release SHA and confirm:

- clean clone installation succeeds;
- frontend production build succeeds;
- review workstation opens at `/app/`;
- one-time model setup verifies pinned sources and prints `READY`;
- normal startup is offline-safe;
- `/health` and `/v1/models` pass;
- one public or synthetic image completes the supported inference path;
- dataset export contains the documented manifest and CSV files;
- no secrets, PHI, model weights, or runtime databases are tracked;
- clinician and operator manuals render successfully.